RUBRIC COMPONENT #8 (6 MARKS)	OFFICIAL CRITERIA: Advanced SQL concepts: procedure/function, trigger, CTE

Advanced Procedural SQL: Procedure, Function, Trigger, CTE

Course: DBMS Mini Project (UE24CS395) • **Institution:** PES University (2026–2027)
Team: Saatvik Gupta (PES1UG24CS395) & Satyam Goyal (PES1UG24CS424)

1. Stored Procedure: sp_create_investigation_case

```
CREATE OR REPLACE PROCEDURE sp_create_investigation_case(
    IN p_case_id VARCHAR(50), IN p_title VARCHAR(200), IN p_description TEXT,
    IN p_case_type VARCHAR(100), IN p_priority VARCHAR(50), IN p_lead_officer_id VARCHAR(50)
) LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO cases (case_id, title, description, case_type, status, priority, lead_officer_id)
    VALUES (p_case_id, p_title, p_description, p_case_type, 'Open', p_priority, p_lead_officer_id);

    INSERT INTO case_activity (case_id, activity_type, title, description, officer_id)
    VALUES (p_case_id, 'CASE_INITIALIZED', 'Case Record Created', 'Initialized with priority: ' || p_priority, p_lead_officer_id);
END; $$;
-- EXECUTION: CALL sp_create_investigation_case('CASE-2026-999', 'Op Neon', 'Crypto ring', 'Cyber', 'High', 'OFF-001');
```

2. User-Defined Function: fn_get_case_summary

```
CREATE OR REPLACE FUNCTION fn_get_case_summary(p_case_id VARCHAR(50)) RETURNS JSONB AS $$
DECLARE v_result JSONB;
BEGIN
    SELECT jsonb_build_object(
        'case_id', c.case_id, 'title', c.title, 'status', c.status,
        'lead_officer', o.full_name || ' (' || o.badge_number || ')',
        'stats', jsonb_build_object('people', COUNT(DISTINCT cp.person_id), 'evidence', COUNT(DISTINCT e.evidence_id))
    ) INTO v_result
    FROM cases c LEFT JOIN officers o ON c.lead_officer_id = o.officer_id
    LEFT JOIN case_people cp ON c.case_id = cp.case_id
    LEFT JOIN evidence e ON c.case_id = e.case_id
    WHERE c.case_id = p_case_id GROUP BY c.case_id, c.title, c.status, o.full_name, o.badge_number;
    RETURN v_result;
END; $$ LANGUAGE plpgsql STABLE;
```

3. Row-Level Audit Trigger: trg_case_activity_audit

```
CREATE OR REPLACE FUNCTION fn_audit_case_state_change() RETURNS TRIGGER AS $$
BEGIN
    IF (OLD.status IS DISTINCT FROM NEW.status) THEN
        INSERT INTO case_activity (case_id, activity_type, title, description, officer_id)
        VALUES (NEW.case_id, 'STATUS_CHANGE', 'Status Updated',
                'Status transitioned from "' || OLD.status || '" to "' || NEW.status || '".', NEW.lead_officer_id);
    END IF;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_case_activity_audit
    AFTER UPDATE OF status, priority ON cases FOR EACH ROW EXECUTE FUNCTION fn_audit_case_state_change();
-- VERIFICATION: Updating cases.status automatically inserts an immutable row into case_activity.
```